Current Progress

This document explains **what each command you ran actually does**, why you ran it, what to expect, and how to verify success.

It is organized in the same order as your script, with short, precise explanations.

> Legend
>
> - **What it does** – one-line summary
> - **Why** – intent/rationale
> - **Side effects** – local/GCP resources created or modified
> - **Verify** – quick check(s)
> - **Notes / pitfalls** – tips, idempotency, common errors

---

0) Local setup – Homebrew & Google Cloud SDK

```
# Update Homebrew
brew update
```

- **What it does:** Refreshes Homebrew's package/cask metadata.
- **Why:** Ensures you install the latest versions/formula definitions.
- **Side effects:** Updates local brew taps; doesn't upgrade installed packages.
- **Verify:** `brew --version` (optional).
- **Notes:** Safe/idempotent.

```
# Install Google Cloud SDK (try cask first; fall back to formula)
brew install --cask google-cloud-sdk || brew install google-cloud-sdk
```

- **What it does:** Installs the Google Cloud SDK (CLI tools: `gcloud`, `gsutil`, etc.). Tries the macOS cask first; if unavailable, falls back to formula.
- **Why:** You need `gcloud`/`gsutil` for all subsequent GCP work.
- **Side effects:** Adds the SDK to your system; config files under `~/.config/gcloud`.
- **Verify:** `gcloud --version`.
- **Notes:** Only one of cask/formula will install; the `||` makes it robust.

```
# Add gcloud to your zsh PATH & completions (handles both install locations)
if [ -f "$(brew --prefix)/Caskroom/google-cloud-sdk/latest/google-cloud-sdk/path.zsh.inc" ]; then
  echo 'source "$(brew --prefix)/Caskroom/google-cloud-sdk/latest/google-cloud-sdk/path.zsh.inc"' >> ~/.zshrc
  echo 'source "$(brew --prefix)/Caskroom/google-cloud-sdk/latest/google-cloud-sdk/completion.zsh.inc"' >> ~/.zshrc
elif [ -f "$(brew --prefix)/share/google-cloud-sdk/path.zsh.inc" ]; then
  echo 'source "$(brew --prefix)/share/google-cloud-sdk/path.zsh.inc"' >> ~/.zshrc
  echo 'source "$(brew --prefix)/share/google-cloud-sdk/completion.zsh.inc"' >> ~/.zshrc
fi
```

- **What it does:** Appends `source ...path.zsh.inc` (PATH) and `...completion.zsh.inc` (tab-completion) to `~/.zshrc`, handling cask vs formula locations.
- **Why:** So your shell finds `gcloud` and offers CLI autocompletion.
- **Side effects:** Edits your shell startup file.
- **Verify:** Reopen terminal or `source ~/.zshrc` then `which gcloud`.
- **Notes:** Appends every time you run it; if repeated, you may have duplicate lines (harmless).

```
# Reload your shell config
source ~/.zshrc
```

- **What it does:** Reloads `~/.zshrc` in the current shell.
- **Why:** Apply PATH/completion changes immediately.
- **Verify:** `gcloud --version`.

```
# Verify
gcloud --version
```

- **What it does:** Prints the installed SDK version and components.
- **Why:** Sanity check after installation.

```
# Sign in to GCP (browser opens)
gcloud auth login
gcloud auth application-default login
```

- **What it does:**
  - `gcloud auth login` authenticates your **user** for `gcloud` commands.
  - `gcloud auth application-default login` creates **Application Default Credentials (ADC)** for client libraries and tools that rely on ADC.
- **Why:** Both user tokens (for gcloud CLI) and ADC (for SDKs/jobs) are needed.
- **Side effects:** Credentials written to `~/.config/gcloud/` and `~/.config/gcloud/application_default_credentials.json`.
- **Verify:** `gcloud auth list` and `gcloud auth application-default print-access-token`.
- **Notes:** If you work across multiple projects, you may later set an ADC **quota project** (you did below).

```
# === EDIT THIS: pick a globally-unique, lowercase project id (letters, digits, h
yphens) ===
export PROJECT_ID="oob-<your-unique-id>"
# Region that supports Cloud Run GPU (L4)
export REGION="us-central1"
echo "PROJECT_ID=$PROJECT_ID  REGION=$REGION"
```

- **What it does:** Defines your working **GCP project id** and **region** in the shell; prints them.
- **Why:** Subsequent commands read these env vars for reproducibility.
- **Side effects:** None in GCP yet.
- **Notes:** You later standardized on `PROJECT_ID="levizion-ai"`.

---

1) Foundational GCP resources – registries, buckets, service accounts, IAM

```
# Artifact Registry (Docker)
gcloud artifacts repositories create bball \\
  --repository-format=DOCKER \\
  --location="$REGION" \\
  --description="Basketball OOB images" || echo "Repo may already exist"
```

- **What it does:** Creates a Docker Artifact Registry repo named `bball` in `us-central1`.
- **Why:** To store container images for Cloud Run/Batch.
- **Side effects:** New AR repo: `us-central1-docker.pkg.dev/<PROJECT_ID>/bball`.
- **Verify:** `gcloud artifacts repositories list --location="$REGION"`.
- **Notes:** You **later** use `AR_REPO=.../levizion`. Make sure either:
  - Create `levizion` repo too, or
  - Update your workflows to push to `bball`.
    Mismatch will cause `docker push`/deploy failures.

```
# Buckets (data + mlflow). Names must be globally unique; this pattern is fine.
gsutil mb -l "$REGION" "gs://${PROJECT_ID}-oob-data" || true
gsutil mb -l "$REGION" "gs://${PROJECT_ID}-mlflow"   || true
```

- **What it does:** Creates two regional GCS buckets: data + MLflow artifacts.
- **Why:** Store datasets and training outputs/metadata.

- **Side effects:** Buckets `gs://levizion-ai-oob-data` and `gs://levizion-ai-mlflow`.
- **Verify:** `gsutil ls -p "$PROJECT_ID"`.
- **Notes:** `|| true` makes it idempotent (skip error if bucket exists). Bucket **location cannot be changed** after creation.

```
# Service accounts
gcloud iam service-accounts create github-deployer --display-name="GitHub deployer" || true
gcloud iam service-accounts create run-infer      --display-name="Cloud Run GPU runtime" || true
gcloud iam service-accounts create jobs-runner    --display-name="Cloud Run Jobs runtime" || true
gcloud iam service-accounts create batch-runner   --display-name="Cloud Batch runtime" || true
```

- **What it does:** Creates 4 service accounts (SAs) for CI/CD and runtimes.
- **Why:** Principle of least privilege per workload.
- **Side effects:** SAs:
  - `github-deployer@<PROJECT_ID>.iam.gserviceaccount.com`
  - `run-infer@...`, `jobs-runner@...`, `batch-runner@...`
- **Verify:** `gcloud iam service-accounts list`.

```
# IAM for deployer (broad but practical for solo MVP; can tighten later)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \\
  --member="serviceAccount:github-deployer@${PROJECT_ID}.iam.gserviceaccount.com" \\
  --role="roles/run.admin"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \\
  --member="serviceAccount:github-deployer@${PROJECT_ID}.iam.gserviceaccount.com" \\
  --role="roles/artifactregistry.writer"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \\
  --member="serviceAccount:github-deployer@${PROJECT_ID}.iam.gservice
```

```
account.com" \\
  --role="roles/iam.serviceAccountUser"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \\
  --member="serviceAccount:github-deployer@${PROJECT_ID}.iam.gservice
account.com" \\
  --role="roles/batch.admin"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \\
  --member="serviceAccount:github-deployer@${PROJECT_ID}.iam.gservice-
account.com" \\
  --role="roles/storage.admin"
```

- **What it does:** Grants the deployer SA project-level roles:
  - **Run Admin** – deploy/manage Cloud Run services.
  - **Artifact Registry Writer** – push images.
  - **Service Account User** – impersonate runtime SAs when needed.
  - **Batch Admin** – submit/manage Batch jobs.
  - **Storage Admin** – create/write/list buckets/objects.
- **Why:** Enable GitHub Actions to build, push, and deploy.
- **Side effects:** Project IAM policy updated.
- **Verify:** `gcloud projects get-iam-policy "$PROJECT_ID" --flatten="bindings[].members" --filter="github-deployer"`.

```
# Bucket access for runtimes
gsutil iam ch "serviceAccount:run-infer@${PROJECT_ID}.iam.gserviceaccoun
t.com:objectViewer"  "gs://${PROJECT_ID}-oob-data"
gsutil iam ch "serviceAccount:jobs-runner@${PROJECT_ID}.iam.gserviceacc
ount.com:objectAdmin"  "gs://${PROJECT_ID}-oob-data"
gsutil iam ch "serviceAccount:batch-runner@${PROJECT_ID}.iam.gserviceac
count.com:objectAdmin" "gs://${PROJECT_ID}-oob-data"
gsutil iam ch "serviceAccount:batch-runner@${PROJECT_ID}.iam.gserviceac
count.com:objectAdmin" "gs://${PROJECT_ID}-mlflow"
```

- **What it does:** Grants object-level permissions on specific buckets:
  - `run-infer` → **objectViewer** on data bucket (read inferences).
  - `jobs-runner` → **objectAdmin** on data bucket (Cloud Run Jobs).
  - `batch-runner` → **objectAdmin** on data + mlflow buckets (training).
- **Why:** Runtime jobs need to read/write datasets and artifacts.
- **Side effects:** Bucket IAM policies updated.
- **Verify:** `gsutil iam get "gs://<bucket>" | jq` (check bindings).

```
# === Verify (paste these outputs) ===
echo "---- gcloud config ----"
gcloud config list --format yaml

echo "---- enabled services (subset) ----"
gcloud services list --enabled | egrep "run|artifact|storage|batch|iam|eventar
c|compute|scheduler|logging|monitoring|sts" || true

echo "---- artifact registry repos ----"
gcloud artifacts repositories list --location="$REGION"

echo "---- buckets ----"
gsutil ls -p "$PROJECT_ID"

echo "---- service accounts ----"
gcloud iam service-accounts list --format="table(email,displayName)"
```

- **What it does:** Prints configuration and confirms resources.
- **Why:** Baseline verification.
- **Notes:** If APIs are missing, enable them: `gcloud services enable run.googleapis.com artifactregistry.googleapis.com batch.googleapis.com storage.googleapis.com compute.googleapis.com logging.googleapis.com iamcredentials.googleapis.com sts.googleapis.com`

```
gcloud auth application-default set-quota-project levizion-ai
```

- **What it does:** Sets the **quota project** used by ADC.
- **Why:** Avoids “quota project not set” errors when libraries call Google APIs.
- **Side effects:** Writes the project number to ADC config.
- **Verify:** `gcloud auth application-default print-token` works; some calls show levizion-ai as the billing/quota project.

---

2) GitHub ↔ GCP Workload Identity Federation (WIF)

```
export GH_OWNER="<your_github_username_or_org>"
export GH_REPO="<your_repo_name>"
export GH_BRANCH="main"
export PROJECT_NUMBER=$(gcloud projects describe levizion-ai --format
="value(projectNumber)")
echo "PROJECT_NUMBER=$PROJECT_NUMBER"
```

- **What it does:** Sets GitHub repo identity and discovers the project number (needed for WIF resource names).
- **Why:** WIF filters which repo/branch can impersonate your deploy SA.

```
# Create the pool
gcloud iam workload-identity-pools create github-pool \\
  --project levizion-ai --location=global \\
  --display-name="GitHub OIDC pool"
```

- **What it does:** Creates a WIF **pool** to trust external identities (GitHub OIDC).
- **Why:** So GitHub Actions can exchange OIDC tokens for short-lived GCP credentials (no JSON key).

```
# Create the OIDC provider for GitHub Actions tokens
gcloud iam workload-identity-pools providers create-oidc github-provider \\
  --project levizion-ai --location=global \\
  --workload-identity-pool=github-pool \\
  --display-name="GitHub provider" \\
```

```
  --issuer-uri="<https://token.actions.githubusercontent.com>" \\
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=ass
ertion.repository,attribute.ref=assertion.ref" \\
  --attribute-condition="assertion.repository=='${GH_OWNER}/${GH_REPO}'
&& assertion.ref=='refs/heads/${GH_BRANCH}'"
```

- **What it does:** Adds a **provider** that accepts GitHub's OIDC tokens, maps claims, and restricts usage to a **single repo + branch**.
- **Why:** Security; only your repo's `main` can assume the SA.
- **Verify:** `gcloud iam workload-identity-pools providers describe ...`.

```
echo "WIF_PROVIDER=projects/${PROJECT_NUMBER}/locations/global/workl
oadIdentityPools/github-pool/providers/github-provider"
```

- **What it does:** Prints the provider resource string for use in GitHub Actions.

```
gcloud iam service-accounts add-iam-policy-binding \\
  github-deployer@levizion-ai.iam.gserviceaccount.com \\
  --role="roles/iam.workloadIdentityUser" \\
  --member="principalSet://iam.googleapis.com/projects/${PROJECT_NUMBE
R}/locations/global/workloadIdentityPools/github-pool/attribute.repository/
${GH_OWNER}/${GH_REPO}"
```

- **What it does:** Allows identities from your pool with attribute `repository=<owner>/<repo>` to **impersonate** the `github-deployer` SA.
- **Why:** This is the bridge from GitHub OIDC → SA credentials.
- **Verify:** `gcloud iam service-accounts get-iam-policy ...` shows the binding.

```
gcloud iam service-accounts get-iam-policy \\
  github-deployer@levizion-ai.iam.gserviceaccount.com \\
  --format=json
```

- **What it does:** Prints the IAM policy for the deployer SA.

- **Why:** Confirm the `workloadIdentityUser` binding is present.

```
echo "GCP_PROJECT_ID=levizion-ai"
echo "GCP_REGION=us-central1"
echo "GCP_WIF_PROVIDER=projects/${PROJECT_NUMBER}/locations/global/
workloadIdentityPools/github-pool/providers/github-provider"
echo "GCP_SERVICE_ACCOUNT_EMAIL=github-deployer@levizion-ai.iam.gse
rviceaccount.com"
echo "AR_REPO=us-central1-docker.pkg.dev/levizion-ai/levizion"
```

- **What it does:** Echoes the values you will store as GitHub **Repository Variables**.
- **Why:** Inputs for GitHub Actions without storing long-lived credentials.
- **Notes:** Ensure the **Artifact Registry repo** `levizion` **exists** or update to `bball`.

---

3) Project defaults & minimal CPU service (API-UI) on Cloud Run

```
# === GCP PROJECT/WIF CONFIG ===
export PROJECT_ID="levizion-ai"
export PROJECT_NUMBER="20311219227"
export REGION="us-central1"
export AR_REPO="us-central1-docker.pkg.dev/${PROJECT_ID}/levizion"
export DATA_BUCKET="gs://${PROJECT_ID}-oob-data"
export MLFLOW_BUCKET="gs://${PROJECT_ID}-mlflow"
export SA_DEPLOYER="github-deployer@${PROJECT_ID}.iam.gserviceaccou
nt.com"
export SA_RUN_INFER="run-infer@${PROJECT_ID}.iam.gserviceaccount.co
m"
export SA_JOBS_RUNNER="jobs-runner@${PROJECT_ID}.iam.gserviceaccou
nt.com"
export SA_BATCH_RUNNER="batch-runner@${PROJECT_ID}.iam.gserviceacc
ount.com"
export GCP_WIF_PROVIDER="projects/${PROJECT_NUMBER}/locations/globa
```

```
l/workloadIdentityPools/github-pool/providers/github-provider"
export GH_OWNER="ry4ntr1"
export GH_REPO="levizion"
export GH_BRANCH="main"
```

- **What it does:** Centralizes all key variables for consistency.

```
# Helpful gcloud defaults (idempotent)
gcloud config set project "$PROJECT_ID"
gcloud config set run/region "$REGION"
gcloud config set artifacts/location "$REGION"
gcloud config set compute/region "$REGION"
gcloud config set compute/zone "${REGION}-a"
```

- **What it does:** Sets default project/regions so you don't need `-project/--region` each time.
- **Side effects:** Updates local gcloud config at `~/.config/gcloud/configurations/`.

```
# From repo root
mkdir -p services/api deploy/dockerfiles .github/workflows
```

- **What it does:** Creates a conventional repo structure.

```
# Minimal FastAPI app
cat > services/api/main.py <<'PY'
from fastapi import FastAPI

app = FastAPI(title="Levizion API-UI (CPU)")

@app.get("/health")
def health():
    return {"ok": True}

@app.get("/")
def root():
```

```
    return {"message": "Levizion OOB MVP: API-UI is up"}
PY
```

- **What it does:** Writes a tiny FastAPI service with `/` and `/health`.
- **Why:** Simple health-checked app to deploy to Cloud Run.

```
# Requirements for this CPU app
cat > requirements.txt <<'REQ'
fastapi
uvicorn[standard]
REQ
```

- **What it does:** App dependencies for the CPU service.

```
# Dockerfile for Cloud Run (CPU)
cat > deploy/dockerfiles/Dockerfile.ui <<'DOCKER'
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y --no-install-recommends build-esse
ntial && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["uvicorn","services.api.main:app","--host","0.0.0.0","--port","8080"]
DOCKER
```

- **What it does:** Containerizes the app for Cloud Run.
- **Notes:** Exposes port 8080 (Cloud Run standard).

```
# GitHub Actions: deploy to Cloud Run (CPU) on push to main
cat > .github/workflows/deploy-api-ui.yml <<'YAML'
```

```
...
YAML
```

- **What it does:** CI/CD pipeline that:
  - Auths via WIF to impersonate `github-deployer`.
  - Builds image and pushes to `$AR_REPO`.
  - Deploys to Cloud Run service `api-ui` (`-allow-unauthenticated`).
- **Verify:** GitHub Actions run passes; service URL responds.

```
git add -A
git commit -m "feat: minimal API-UI + Cloud Run deploy workflow"
git push origin main
```

- **What it does:** Commits and pushes; **triggers** the workflow.

---

4) Set GitHub repo-level variables & re-deploy

```
# === GCP PROJECT/WIF CONFIG ===
export PROJECT_ID="levizion-ai"
...
export GCP_WIF_PROVIDER="projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/github-pool/providers/github-provider"

# GitHub repo identity (DO NOT export GH_REPO here — it conflicts with gh CLI)
export GH_OWNER="ry4ntr1"
export GH_REPO_NAME="levizion"
export GITHUB_REPO="${GH_OWNER}/${GH_REPO_NAME}"
...
unset GH_REPO
```

- **What it does:** Avoids `GH_REPO` collision with `gh` CLI while assembling `<owner>/<name>`.

```
# (Re)set the variables explicitly for this repo
gh variable set GCP_PROJECT_ID --repo "$GITHUB_REPO" --body "$PROJECT_ID"
gh variable set GCP_REGION --repo "$GITHUB_REPO" --body "$REGION"
gh variable set GCP_WIF_PROVIDER --repo "$GITHUB_REPO" --body "$GCP_WIF_PROVIDER"
gh variable set GCP_SERVICE_ACCOUNT_EMAIL --repo "$GITHUB_REPO" --body "$SA_DEPLOYER"
gh variable set AR_REPO --repo "$GITHUB_REPO" --body "$AR_REPO"
```

- **What it does:** Stores non-secret config as **Repository Variables** for Actions.
- **Why:** So workflows can reference `${{ vars.* }}`.
- **Verify:** `gh variable list --repo "$GITHUB_REPO"`.

```
git commit --allow-empty -m "ci: redeploy api-ui with variables"
git push origin main
```

- **What it does:** Kicks off a new deployment run without code changes.

```
gcloud run services describe api-ui --region "$REGION" --format='value(status.url)'
curl -s $(gcloud run services describe api-ui --region "$REGION" --format='value(status.url)')/health
```

- **What it does:** Fetches service URL and probes `/health`.
- **Expect:** `{"ok": true}`.

---

5) Harden the workflow trigger & redeploy

```
# Replace the workflow with one that triggers on any push to main + manual
cat > .github/workflows/deploy-api-ui.yml <<'YAML'
...
YAML
```

```
git add .github/workflows/deploy-api-ui.yml
git commit -m "ci: broaden trigger + set project explicitly for api-ui deploy"
git push origin main
```

- **What it does:** Ensures the deploy job always sets `gcloud` project from repo variables and can be run manually (`workflow_dispatch`).

```
gcloud run services describe api-ui --region "$REGION" --format='value(stat
us.url)'
curl -s $(gcloud run services describe api-ui --region "$REGION" --format='v
alue(status.url)')/health
```

- **Verify:** Service remains healthy after redeploy.

---

6) GPU readiness & minimal GPU Cloud Run service

```
gcloud compute regions describe "$REGION" \\
  --format="table(quotas.metric,quotas.limit,quotas.usage)" | egrep -i "NVIDIA
|GPU|ACCEL"
```

- **What it does:** Shows relevant GPU quotas in `us-central1`.
- **Why:** Confirm available capacity/limits prior to GPU deployments.

```
# 9.1 – Minimal GPU app
mkdir -p services/infer deploy/dockerfiles .github/workflows

cat > services/infer/main.py <<'PY'
from fastapi import FastAPI
import os, json

app = FastAPI(title="Levizion Inference (GPU)")

def gpu_status():
```

```
    info = {"cuda_visible": os.environ.get("CUDA_VISIBLE_DEVICES", ""),
            "nvidia_env": os.environ.get("NVIDIA_VISIBLE_DEVICES", "")}
    try:
        import torch
        cuda_ok = torch.cuda.is_available()
        devices = []
        if cuda_ok:
            count = torch.cuda.device_count()
            for i in range(count):
                devices.append(torch.cuda.get_device_name(i))
        info.update({"torch": True, "cuda_available": cuda_ok, "devices": device
s})
    except Exception as e:
        info.update({"torch": False, "error": str(e)})
    return info

@app.get("/health")
def health():
    return {"ok": True}

@app.get("/gpu")
def gpu():
    return gpu_status()
PY
```

- **What it does:** GPU-aware FastAPI that reports CUDA/Torch status at `/gpu`.

```
# 9.2 – GPU Dockerfile (installs CUDA-enabled PyTorch)
cat > deploy/dockerfiles/Dockerfile.gpu <<'DOCKER'
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

# Basic deps
RUN apt-get update && apt-get install -y --no-install-recommends \\
```

```
    build-essential && rm -rf /var/lib/apt/lists/*

# Install FastAPI + CUDA-enabled PyTorch wheels
RUN pip install --no-cache-dir fastapi uvicorn[standard]
# PyTorch CUDA 12.1 wheels (keep versions reasonably current)
RUN pip install --no-cache-dir --index-url <https://download.pytorch.org/whl/
cu121> \\
    torch torchvision torchaudio

COPY . .
EXPOSE 8080
CMD ["uvicorn","services.infer.main:app","--host","0.0.0.0","--port","8080"]
DOCKER
```

- **What it does:** Builds a GPU-enabled container using prebuilt CUDA 12.1 PyTorch wheels.
- **Notes:** No system CUDA libraries required; Cloud Run GPU provides drivers.

```
# 9.3 – GitHub Actions workflow for GPU service
cat > .github/workflows/deploy-infer-gpu.yml <<'YAML'
...
YAML

git add -A
git commit -m "feat: minimal GPU inference service + deploy workflow"
git push origin main
```

- **What it does:** CI/CD for `infer-gpu` service with `-gpu 1 --gpu-type nvidia-l4`, `-cpu 4 --memory 16Gi`, `-concurrency 1`, and runtime SA `run-infer`.
- **Verify:** GitHub run passes.

```
# Get the GPU service URL
gcloud run services describe infer-gpu --region "$REGION" --format='value(s
tatus.url)'
```

```
# Health
curl -s $(gcloud run services describe infer-gpu --region "$REGION" --format
='value(status.url)')/health

# GPU status
curl -s $(gcloud run services describe infer-gpu --region "$REGION" --format
='value(status.url)')/gpu | jq .
```

- **Expect:** `/gpu` returns JSON with `torch: true`, `cuda_available: true`, and `devices: ["NVIDIA L4", ...]`.

---

7) Dataset upload to GCS (Roboflow YOLOv8 export)

```
export PROJECT_ID="levizion-ai"
export REGION="us-central1"
export DATA_BUCKET="gs://${PROJECT_ID}-oob-data"
export LOCAL_DATASET="$HOME/Datasets/roboflow_yolov8"   # ← EDIT THI
S
export DATASET_NAME="roboflow_bball_yv8_v1"
export GCS_DATASET_URI="${DATA_BUCKET}/datasets/${DATASET_NAME}"
```

- **What it does:** Points to your local dataset and the destination prefix in GCS.

```
# Upload
gsutil -m rsync -r "$LOCAL_DATASET" "$GCS_DATASET_URI"
```

- **What it does:** Recursively mirrors local dataset → `gs://.../datasets/<name>`.
- **Verify:** `gsutil ls -r "$GCS_DATASET_URI" | sed -n '1,50p'`.
- **Notes:** Your training script expects a **`data.yaml`** at the root of this GCS folder.

---

8) Training pipeline (Ultralytics YOLO) via Cloud Batch (L4)

```
# 1) Training script
mkdir -p pipeline deploy/dockerfiles .github/workflows

cat > pipeline/train_detector.py <<'PY'
# (script omitted here for brevity – it downloads dataset from GCS, trains YOL
O, exports ONNX, uploads artifacts)
PY
```

- **What it does:**
  - **Downloads** dataset from `-data_uri` (GCS) into `/workspace/dataset`.
  - Runs `YOLO(...).train(...)` on GPU (`device=0`), saving under `/workspace/outputs`.
  - **Exports** best weights to ONNX.
  - **Writes** a `summary.json`.
  - **Uploads** the whole run dir to `-out_uri` (GCS).
- **Verify:** After job completes, `gs://.../models/<run_name>/` contains `weights/best.pt`, `.onnx`, `summary.json`, logs, etc.

```
# 2) CUDA-enabled training image
cat > deploy/dockerfiles/Dockerfile.train <<'DOCKER'
FROM pytorch/pytorch:2.3.1-cuda12.1-cudnn8-runtime
...
RUN pip install --no-cache-dir ultralytics google-cloud-storage pyyaml openc
v-python-headless
COPY . .
CMD ["python","pipeline/train_detector.py","--help"]
DOCKER
```

- **What it does:** Defines the training container with CUDA/PyTorch and dependencies.

```
# 3) GitHub Action: build train image + submit Cloud Batch job (GPU L4)
cat > .github/workflows/train-batch.yml <<'YAML'
```

```
...
YAML

git add -A
git commit -m "feat: Cloud Batch GPU training pipeline (Ultralytics YOLO)"
git push origin main
```

- **What it does:** Adds a manual workflow (`workflow_dispatch`) that:
  - Builds/pushes the training image to `$AR_REPO`.
  - Submits a **Cloud Batch** job:
    - Machine: `g2-standard-8` with **1× L4**
    - `installGpuDrivers: true` (Batch installs drivers)
    - Logs go to **Cloud Logging**
  - Passes workflow inputs to your training script.

```
# List jobs (you should see train-detector-<run_id>)
gcloud batch jobs list --location us-central1

# Replace <JOB_NAME> with the job printed by the Action
gcloud batch jobs describe <JOB_NAME> --location us-central1 --format='val
ue(status.state)'
# Repeat until it shows: SUCCEEDED (or FAILED)

# Tail logs
gcloud logging read 'resource.type="batch_job" AND resource.labels.job_id
="<JOB_NAME>"' --limit 50 --format json | jq '.[].textPayload' | sed -n '1,120p'
```

- **What it does:** Monitors job state and fetches logs.
- **Expect:** Final state `SUCCEEDED`; logs show dataset download, training progress, ONNX export, and GCS upload.

---

9) Local dataset prep & re-run with explicit SA for Batch

```
# 1) Create a tidy place for datasets
mkdir -p "$HOME/Datasets"

# 2) See if the zip is in Downloads
ls -lh ~/Downloads | egrep -i 'roboflow|yolo|basketball'

# 3) Unzip into canonical folder (EDIT filename)
unzip -q "$HOME/Downloads/<YOUR_ROBOFLOW_EXPORT>.zip" -d "$HOM
E/Datasets/roboflow_yolov8"

# 4) Verify expected structure
ls -R "$HOME/Datasets/roboflow_yolov8" | sed -n '1,80p'
```

- **What it does:** Prepares local dataset folder with `data.yaml`, `train/`, `val/`, etc.

```
export PROJECT_ID="levizion-ai"
export DATA_BUCKET="gs://${PROJECT_ID}-oob-data"
export LOCAL_DATASET="$HOME/Datasets/roboflow_yolov8"
export DATASET_NAME="roboflow_bball_yv8_v1"
export GCS_DATASET_URI="${DATA_BUCKET}/datasets/${DATASET_NAME}"

# Upload (mirrors local dir to GCS folder)
gsutil -m rsync -r "$LOCAL_DATASET" "$GCS_DATASET_URI"

# Verify on GCS (first 50 lines list)
gsutil ls -r "$GCS_DATASET_URI" | sed -n '1,50p'
```

- **What it does:** Uploads/verifies dataset on GCS.

```
# Patch train-batch.yml to add the service account for Batch tasks
awk '
/"taskSpec": {/ && !s { print; print "            \\"serviceAccount\\": { \\"email\\":
```

```
\\"batch-runner@${{ vars.GCP_PROJECT_ID }}.iam.gserviceaccount.com\\"
},"; s=1; next }1
' .github/workflows/train-batch.yml > .github/workflows/train-batch.yml.patch
ed && mv .github/workflows/train-batch.yml.patched .github/workflows/train-
batch.yml

git add .github/workflows/train-batch.yml
git commit -m "ci: run Cloud Batch jobs as batch-runner SA for GCS access"
git push origin main
```

- **What it does:** Injects a `serviceAccount` field into the Batch **taskSpec** so tasks run as `batch-runner@...`.
- **Why:** Ensures the job can read/write GCS (you granted objectAdmin earlier).
- **Verify:** In the job details JSON, `serviceAccount.email` matches `batch-runner@...`.

```
# If you have gh CLI and are authenticated:
gh workflow run "Train Detector (Cloud Batch GPU)" \\
  -f dataset_gcs_uri="gs://levizion-ai-oob-data/datasets/roboflow_bball_yv8_v
1" \\
  -f out_gcs_uri="gs://levizion-ai-oob-data/models/detector/run1" \\
  -f epochs="30" \\
  -f imgsz="1280" \\
  -f base_model="yolov8s.pt"

# Watch
gh run watch --exit-status || gh run view --log
```

- **What it does:** Triggers training with specific inputs and watches Action logs.

```
# Monitor from GCP
gcloud batch jobs list --location us-central1
export JOB_NAME="<train-detector-...>"
gcloud batch jobs describe "$JOB_NAME" --location us-central1 --format='va
lue(status.state)'
```

```
gcloud batch tasks list --job "$JOB_NAME" --location us-central1
gcloud logging read \\
  'resource.type="batch_job" AND resource.labels.job_id="'$JOB_NAME'"' \\
  --limit 120 --format json | jq -r '.[].textPayload'
```

- **What it does:** Confirms job state, tasks, and retrieves logs.

10) Resource Map (what exists now)

Category	Resource	Name / Path
Project	ID / Number	`levizion-ai` / `20311219227`
Region	Primary	`us-central1`
Artifact Registry	Docker repos	`bball` (created), `levizion` (referenced by workflows)
Buckets	Data / MLflow	`gs://levizion-ai-oob-data`, `gs://levizion-ai-mlflow`
Service Accounts	CI/CD	`github-deployer@levizion-ai.iam.gserviceaccount.com`
	Runtimes	`run-infer@...`, `jobs-runner@...`, `batch-runner@...`
WIF	Pool / Provider	`github-pool` / `github-provider` (restricted to `ry4ntr1/levizion@refs/heads/main`)
Cloud Run	CPU service	`api-ui` (public)
	GPU service	`infer-gpu` (L4, public)
Batch	Jobs	`train-detector-<run_id>` (L4, g2-standard-8)
GitHub	Repo vars	`GCP_PROJECT_ID`, `GCP_REGION`, `GCP_WIF_PROVIDER`, `GCP_SERVICE_ACCOUNT_EMAIL`, `AR_REPO`

11) Quick Verify Cheat-Sheet

- Config: `gcloud config list --format yaml`
- APIs: `gcloud services list --enabled | egrep 'run|artifact|storage|batch|iam|sts|compute'`
- AR repos: `gcloud artifacts repositories list --location us-central1`
- Buckets: `gsutil ls -p levizion-ai`

- SAs: `gcloud iam service-accounts list --format="table(email,displayName)"`
- WIF binding: `gcloud iam service-accounts get-iam-policy github-deployer@levizion-ai.iam.gserviceaccount.com --format=json`
- Cloud Run URLs: `gcloud run services describe api-ui --region us-central1 --format='value(status.url)'gcloud run services describe infer-gpu --region us-central1 --format='value(status.url)'`
- Health: `curl -s "$API_UI_URL/health"` ; `curl -s "$INFER_GPU_URL/gpu" | jq .`
- Batch jobs: `gcloud batch jobs list --location us-central1`

---

12) Common Pitfalls & Fixes

- **Artifact Registry repo mismatch**

  Workflows use `.../levizion` but you created `bball`.

  - Create `levizion`: `gcloud artifacts repositories create levizion --repository-format=DOCKER --location=us-central1`
  - **OR** change GitHub repo variable `AR_REPO` to `.../bball`.

- **APIs not enabled**

  If you see `API not enabled` errors, run:

  `gcloud services enable run.googleapis.com artifactregistry.googleapis.com batch.googleapis.com storage.googleapis.com compute.googleapis.com logging.googleapis.com iamcredentials.googleapis.com sts.googleapis.com`

- **ADC quota project missing**

  Errors from client libraries about quota/billing:

  `gcloud auth application-default set-quota-project levizion-ai`.

- **Insufficient GPU quota**

  Region lacks L4 quota. Request quota increase or use a supported region.

- **Permissions to read/write GCS in Batch**

  Make sure `batch-runner@...` has **objectAdmin** on your buckets *and* the Batch job/task uses that SA.

- **GH CLI collisions**

  Unset `GH_REPO` before using `gh` to avoid ambiguity: `unset GH_REPO`.

---

13) Cleanup (optional)

- Cloud Run services: `gcloud run services delete api-ui --region us-central1gcloud run services delete infer-gpu --region us-central1`
- Batch jobs (history): `gcloud batch jobs delete <JOB_NAME> --location us-central1`
- Artifact Registry images: `gcloud artifacts docker images list us-central1-docker.pkg.dev/levizion-ai/<repo>`
- Buckets: `gsutil -m rm -r gs://levizion-ai-oob-data/**` (irreversible) `gsutil rb gs://levizion-ai-oob-data` (if empty)

You now have:

1. A minimal **CPU** API deployed to Cloud Run.
2. A minimal **GPU** inference service on L4.
3. A **Cloud Batch** training pipeline that trains YOLO on GCS data and uploads artifacts back to GCS.
4. Secure **GitHub OIDC→WIF** CI that can build, push, and deploy without JSON keys.